D Devloft
TECHNOLOGIES

Developer Guide

TECHNICAL SPECIFICATION

DATE

25 October 2026

PROJECT

Deep Fitness Mobile App

8 modules · Flutter · Firebase · Android & iOS

Flutter

Dart · Firebase · Cloud Functions

01 OVERVIEW

The Deep Fitness app is a Flutter application written in Dart, running on Android and iOS from one codebase, with Firebase as the backend. Gym staff manage content from a web dashboard built in Flutter Web against the same Firebase project.

There is no custom API server. The app reads and writes Firestore directly, and Firestore security rules are the access control. A small set of Cloud Functions covers the work a client cannot be trusted to do: scheduled jobs, push dispatch and privileged writes.

The member's path through the app

1. **Sign in** with phone OTP or email, creating a member document on first login.
2. **Assigned plan**: staff assign a workout plan; the member sees it on the home screen.
3. **Exercise by exercise**: each exercise unlocks only when the previous one is complete or skipped.
4. **Completion log**: every completion writes a timestamped record, which drives all reports.
5. **Classes and events**: browse class videos, book a live class, RSVP to gym events.

Roles: **member** (the app) and **admin** and **trainer** (the dashboard). Brand palette for the UI is the gym's own; set it once in the theme file.

02 ARCHITECTURE & TECH STACK

One Flutter codebase targets Android, iOS and Web. Items marked *assumed* are not fixed by the quotation; confirm or change them before development starts.

LAYER	CHOICE	NOTES

Language	Dart (current stable)	Null safety on; one codebase for app and dashboard
Framework	Flutter (current stable channel)	Android, iOS and Flutter Web for the dashboard
State management	Riverpod *(assumed)*	Providers per feature; no global singletons
Routing	go_router *(assumed)*	Declarative routes with an auth redirect guard
Database	Cloud Firestore	Document store; security rules are the access control
Auth	Firebase Authentication	Phone OTP and email/password
File storage	Firebase Cloud Storage	Exercise and event photos, resized before upload
Server logic	Cloud Functions (TypeScript)	Scheduled jobs, push dispatch, privileged writes
Push	Firebase Cloud Messaging	Android and APNs for iOS
Class videos	External links (YouTube or Vimeo)	Not hosted; the app stores the video ID and embeds the player
Analytics & errors	Firebase Analytics, Crashlytics	Enabled in release builds only
Dashboard	Flutter Web on Firebase Hosting	Same models and repositories as the app

Request path

Widget, provider, repository, Firebase SDK, Firestore rules, document. Every write goes through a repository so model mapping and validation sit in one place. Widgets never touch `FirebaseFirestore.instance` directly.

> Firestore rules are the real access control. The app hides what a member cannot use, but anything the rules allow is reachable with the SDK and a token, so no rule may be skipped for convenience.

03 PROJECT STRUCTURE & SETUP

Feature-first layout: each module is a folder with the same internal shape, so a developer who has read one has read them all.

```
lib/
  main.dart                  # bootstrap, Firebase init, theme, router
  app/
    router.dart              # go_router routes + auth redirect
    theme.dart               # colors, typography, component themes
  core/
    models/                  # shared models, Firestore converters
    repositories/            # base repository, pagination, error mapping
    services/                # auth, messaging, storage, analytics
    widgets/                 # shared buttons, cards, empty and error states
    utils/                   # formatters, validators, date helpers
  features/
    auth/
    profile/
    workout_plan/
      data/                  # repository + Firestore queries
      domain/                # models, enums, business rules
      presentation/          # screens, widgets, providers
    workout_report/
    events/
    group_classes/
    notifications/
  admin/                     # Flutter Web dashboard screens
functions/                   # Cloud Functions (TypeScript)
firestore.rules
storage.rules
```

Setup

1. Install the Flutter SDK (current stable) and run `flutter doctor` until it is clean.
2. Clone the repository and run `flutter pub get`.
3. Install the FlutterFire CLI and run `flutterfire configure` to generate `firebase_options.dart` for each platform.
4. Place `google-services.json` (Android) and `GoogleService-Info.plist` (iOS) in their platform folders. Neither is committed.
5. Run the emulator suite with `firebase emulators:start` for local work, and `flutter run --dart-define=USE_EMULATOR=true`.
6. Deploy rules with `firebase deploy --only firestore:rules,storage`.

Environments

Two Firebase projects: `deepfitness-dev` and `deepfitness-prod`. Flavors select between them at build time; a developer never points a debug build at production data.

04 CONVENTIONS

These rules apply to every module. They keep the data model predictable and make the app's promises (soft delete, ordering, audit) true everywhere.

- **Models over maps.** Every collection has a Dart model with `fromFirestore` and `toFirestore`, registered through `withConverter`. No raw `Map<String, dynamic>` above the repository layer.
- **Repositories only.** Widgets and providers call repositories; only repositories call the Firebase SDK.
- **Base fields.** Every document carries `id`, `createdAt`, `updatedAt`, `createdBy` and `deletedAt`, written with `FieldValue.serverTimestamp()` — never the device clock.
- **Soft delete.** Deleting sets `deletedAt`. Default queries filter it out. Nothing is hard-deleted through the app.
- **Defensive reads.** Documents written by older app versions may lack newer fields; every model defaults a missing field rather than throwing. This matters because the app is already live.
- **Pagination.** List queries use `limit` with `startAfterDocument`, never a full collection read.
- **Enums for status.** Statuses are Dart enums stored as strings, with transitions checked in one place.
- **Errors.** Repositories catch `FirebaseException` and return a typed failure; screens show a message, never a raw exception.
- **Naming.** Collections are plural `snake_case` (`workout_plans`), files `snake_case.dart`, classes `UpperCamelCase`.
- **Formatting.** `dart format` and `flutter analyze` clean before every merge.

05 AUTHENTICATION & ACCESS

Firebase Authentication handles sign-in. A member document is created on first login and keyed by the Firebase UID.

- Phone OTP as the primary method, email and password as a fallback.
- Role is a **custom claim** (`member`, `trainer`, `admin`), set by a Cloud Function. It is never a field the client can write.

- The router redirects on auth state: signed out to login, signed in without a profile to onboarding, otherwise to home.
- A lapsed membership is a state on the member document, not a sign-out. The app shows a renewal screen and keeps read access to the member's own history.

Firestore rules, in outline

```
function isSignedIn()   { return request.auth != null; }
function isAdmin()      { return request.auth.token.role == 'admin'; }
function isSelf(uid)    { return request.auth.uid == uid; }

match /members/{uid} {
  allow read:   if isSelf(uid) || isAdmin();
  allow update: if isSelf(uid) && !changesProtectedFields();
  allow create, delete: if isAdmin();
}

match /assigned_plans/{planId} {
  allow read:   if isSelf(resource.data.memberId) || isAdmin();
  allow create, delete: if isAdmin();
  // a member may only tick progress on their own plan
  allow update: if isSelf(resource.data.memberId)
                && onlyProgressFieldsChanged();
}

match /exercises/{id} {
  allow read:  if isSignedIn();
  allow write: if isAdmin();
}
```

> Write the rules alongside each feature, not at the end, and test them in the emulator with the rules unit-test library. Rules written last are rules written loosely.

06 DATA MODEL

Firestore is a document store, so the model is shaped by the screens that read it rather than by normalisation. Two decisions matter most.

- **Snapshot, don't reference.** An assigned plan copies the exercise details it needs at assignment time. If it referenced the library, an admin editing "Squat" would retroactively change every member's completed history and break the reports.
- **One log, many reports.** Every completion writes one `completion_logs` document. Completion rate, streaks and volume all derive from it, so no report needs its own storage.

COLLECTION	KEY FIELDS	NOTES
`members`	name, phone, email, photoUrl, joinedAt, membershipStatus, expiryDate, fcmTokens[]	Document id is the Firebase UID
`exercises`	name, description, photoUrls[], muscleGroup, defaultSets, defaultReps	The library; admin-managed
`workout_plans`	title, description, exerciseRefs[] (ordered)	Reusable template
`assigned_plans`	memberId, planId, title, exercises[] (snapshot), currentIndex, status, assignedAt	One per member per plan
`completion_logs`	memberId, assignedPlanId, exerciseName, setsDone, skipped, skipReason, completedAt	Append-only; the source for every report
`events`	title, description, imageUrl, startsAt, venue	Public to signed-in members
`event_rsvps`	eventId, memberId, createdAt	One document per member per event
`group_classes`	title, description, videoProvider, videoId, thumbnailUrl, durationMins	Video is linked, never uploaded
`class_sessions`	classId, instructor, startsAt, capacity, bookedCount, waitlistCount	A scheduled run of a class
`bookings`	sessionId, memberId, status (booked, waitlisted, cancelled), position, createdAt	Written through a transaction
`announcements`	title, body, audience, sentAt, sentBy	Mirrors what was pushed

Indexes

Compound queries need declared indexes. Keep `firestore.indexes.json` in the repository and deploy it; do not create indexes by clicking the console error link, or the next environment will be missing them.

07 MODULE REFERENCE

MEMBER ACCESS

01

Authentication & Member Profile

Firebase Auth plus the member document every other module reads.

- Phone OTP and email sign-in; profile created on first login
- Profile edit writes only the member's own document, enforced by rules
- Photo upload resized on device before it reaches Cloud Storage
- FCM token saved to `fcmTokens[]` on login and removed on logout

TRAINING

02

Workout Plans

The largest module. Assignment, sequential unlock and completion logging.

- Exercise library and plan builder are admin-side (dashboard)
- Assignment snapshots exercise details onto `assigned_plans`
- `currentIndex` drives the unlock: an exercise is locked when its index exceeds it
- Completing or skipping an exercise writes a log and advances `currentIndex` in one batch
- Skip requires a reason and still advances, so a member is never stuck
- Unlock progress persists across sessions; it does not reset daily

03

Workout Reports

Computed on device from the completion log over the selected range.

- Range selector: week, month or custom, as a single Firestore query on `completedAt`
- Completion rate, streaks and sets completed derived in Dart, not stored
- Charts drawn with a Flutter charting package; no server round trip
- Shareable summary rendered as an image from the widget tree

GYM ACTIVITY

04

Events

Upcoming gym events with RSVP.

- List query filters `startsAt >= now`, ordered ascending, paginated
- RSVP creates one document; the rules allow a member only their own
- A scheduled function sends the reminder push the day before

05 Group Classes & Booking

On-demand videos, plus booking a place in a live session.

- Videos are external links; store `videoProvider` and `videoId`, build the embed URL in the app
- Player is embedded in-app so a member is not thrown out to YouTube
- **Booking runs in a Firestore transaction**: read `bookedCount` against `capacity`, then write the booking and the counter together, so two members tapping at once cannot oversell the last place
- Full session adds a waitlisted booking with a position
- Cancelling frees the slot and promotes the first waitlisted member, in the same transaction, then pushes them a notification
- Admin view lists bookings per session

06 Notifications

Broadcast announcements and automated reminders over FCM.

- Admin composes in the dashboard; a Cloud Function fans out to tokens or a topic
- Every push is mirrored into `announcements` so the in-app list survives a dismissed notification
- Invalid tokens are pruned from `fcmTokens[]` on send failure
- iOS requires the APNs key uploaded to Firebase and the push capability enabled in Xcode

PLATFORMS

07 iOS App

The same Dart codebase, released to the App Store.

- Platform setup: APNs key, push capability, Info.plist usage strings for camera and photos
- Phone-auth reCAPTCHA fallback configured with the URL scheme
- Layout checked at iPhone SE through Pro Max widths, and with the notch safe areas
- Apple sign-in is required by App Store policy if other social logins are added later

ADMINISTRATION

08 Admin Dashboard (Flutter Web)

Same models and repositories as the app, deployed to Firebase Hosting.

- Manage the exercise library, plans and assignments
- Create events, classes and class sessions
- Member list with workout history
- Compose and send announcements
- Admin-only by custom claim, enforced in rules and not merely hidden in the UI

08 CLOUD FUNCTIONS

Written in TypeScript, kept to the work a client cannot be trusted to do. Everything else stays client-direct.

FUNCTION	TRIGGER	DOES
`onMemberCreate`	Auth create	Creates the member document and sets the default role claim
`setUserRole`	Callable (admin)	Sets a custom claim; the only way a role changes
`sendAnnouncement`	Callable (admin)	Fans out an FCM message and mirrors it into `announcements`
`onBookingCancelled`	Firestore write	Promotes the first waitlisted member and notifies them
`dailyReminders`	Scheduled, daily	Class reminders, event reminders, membership-expiry warnings
`inactivityNudge`	Scheduled, weekly	Pushes members with no completion log in the last 10 days

Scheduled functions require the Blaze plan. Set a budget alert on the project at setup; the usual cause of an unexpected bill is a function that writes a document which re-triggers the same function.

09 TESTING, RELEASE & HANDOVER

Testing

- Unit tests for the rules that matter: unlock advance, skip, booking capacity, report maths.
- Firestore rules tests in the emulator: for each role, what is allowed and what must be denied.
- Widget tests for the plan screen states: locked, active, complete, skipped, empty.
- Integration test of the full path: sign in, assigned plan, complete, report.
- Device testing on a spread of real Android handsets and iPhone sizes, not only emulators.
- `flutter analyze` and `flutter test` clean before every merge.

Release

1. Bump `version` in `pubspec.yaml`; the build number must increase on every store upload.
2. Android: build an app bundle, upload to Play Console, release with a **staged rollout** (10%, 50%, 100%).
3. iOS: archive, upload to App Store Connect, submit for review. Expect a slower review and a possible first rejection.
4. Deploy rules, indexes and functions together with the release, and deploy rules *before* the client that depends on them.
5. Keep the previous bundle so a rollback is possible.

> The app is already live. Data changes go in additively — new fields alongside old ones — and every new field has a default, because an older build will keep reading these documents for weeks after release.

Security checklist

- Rules deny by default; no collection left open to `allow read, write: if true`.
- Roles are custom claims, never client-written fields.
- Storage rules validate content type and size.
- No service-account key or API secret in the repository; functions config holds secrets.
- Crashlytics and Analytics off in debug builds.

Handover

On completion the client receives the full source code, the Firebase project ownership, the store listings, a written user guide, this developer guide and a training session for gym staff. Anything outside the agreed modules is a separate change and should not be added to these modules ad hoc.